

全自動化 AI SDLC

Workflow 與知識庫閉環

課程對象：Workflow 與知識庫閉環
總時數：1 天 × 3 小時
完課條件：完成結業挑戰紀錄表、全流程 7 步驟皆已執行

每日主題

- 全自動化 AI SDLC (3)

全自動化 AI SDLC — Workflow 與知識庫閉環

能夠執行從 ADO 工單到 PR 的完整 AI 全自駕流程，並建立知識庫閉環。

完整的 AI SDLC 概念

- 學習目標：
- 能夠在白板上畫出 AI 全自駕 SDLC 的六大環節流程圖
 - 能夠標示 A2P 安全審批的插入點
 - 能夠區分必須審批 / 建議審批 / 不需審批的操作

AI 全自駕 SDLC 流程圖

六個環節形成完整自動化流水線，每個環節由 AI Agent 執行，最後經 Wiki 動態同步形成知識庫閉環。

- 1. 工單 (ADO) — AI 讀取 Azure DevOps 工單，解析需求與 Acceptance Criteria
- 2. 分析 (Wiki) — AI 查詢技術文件和架構決策，避免重複踩坑
- 3. 開發 (TDD) — AI 以測試驅動方式完成功能實作 ● → ● → ♻️
- 4. 測試 (CI) — AI 執行單元/整合測試，產出覆蓋率報告
- 5. PR Review — AI 自己做 Code Review 後再發 PR
- 6. 文件同步 — AI 自動更新 Wiki，形成知識庫閉環

Note: A2P 安全審批：● 合併 PR / ● 部署到正式環境 / ● 修改資料庫 schema — 必須人類批准。● 新增依賴 / ● 變更 API — 建議批准。● 更新文件 / ● 新增測試 — 不需審批。

A2P (Agent-to-Person) 安全審批模型

人類始終握有最終否決權。AI 負責執行，人類負責批准破壞性操作。這是信任但驗證的實作。

- ● **必須審批** — 合併 PR 到 main branch、部署到 staging/production、修改資料庫 schema、變更安全設定
- ● **建議審批** — 新增依賴套件、變更 API 介面、修改共用模組
- ● **不需審批** — 更新文件、新增測試、Code formatting

任務：

- □ 觀察六環節流程圖 (ADO → 分析 → 開發 → 測試 → PR → Wiki 更新)
- □ 找出循環回饋的位置 (文件同步如何回頭影響工單分析)
- □ 標示 A2P 必須審批 vs 建議審批 vs 不需審批的操作

常見疑問：

- Q: 為什麼每個破壞性操作都要人類批准？ A: AI 可能會錯誤判斷或引入非預期的變更。對正式環境的任何變更，必須有人類最終確認這是業界共識。
- Q: A2P 會拖慢流程嗎？ A: 不會。大多數環節 AI 全自動，只有破壞性操作需要停頓。這也是與一般自動化的核心差異。



AI 全自駕 SDLC 六環節流程圖

Demo — The Finale

學習目標：

- 能夠在看過示範後敘述「一鍵啟動 8 分鐘全流程」的步驟
- 能夠指出 Demo 中出現的兩個 A2P 審批點

8 分鐘完成從工單到 PR 的全流程

以下為 AI 完整執行 8 步驟的過程。特別注意：A2P 審批出現在「更新 Wiki」（Step 7/8）和「發 PR」（Step 8/8）兩個環節。

步驟動作耗時（AI）傳統做法

Step 1/8 讀取工單 #12345（ADO）~30 秒手動打開 ADO 網頁

Step 2/8 查詢 Wiki（訂單服務架構 + 連線池設定）~1 分鐘搜尋文件庫

Step 3/8 分析根因（20 instance × 10 = 200 > 100）~1 分鐘需要經驗判斷

Step 4/8 TDD 開發（ →  → ）~3 分鐘 30-60 分鐘

Step 5/8 執行測試（mvn test + jacoco）~1 分鐘手動跑 CI

Step 6/8 AI Code Review ~1 分鐘人肉 code review

Step 7/8 更新 Wiki（需 A2P 審批 ）~30 秒手動更新文件

Step 8/8 發 PR（需 A2P 審批 ）~1 分鐘建立 branch + 發 PR

Note: ✨ 總耗時：約 8 分鐘（傳統流程約 2-4 小時）。效率提升 15-30 倍。

任務：

- □ 觀察 AI 執行 8 步驟的完整過程（不需動手，先理解流程）
- □ 記錄每個步驟耗時與對應的傳統做法耗時
- □ 觀察 A2P 審批點在哪兩個步驟出現



Demo 8 步驟執行過渡畫面

六大環節詳解

學習目標：

- 能夠針對每個環節說出對應的工具/Skill 名稱
- 能夠說出每個環節在流程中的具體角色

環節一：ADO 工單綁定

透過 `ado-devops` Skill，AI 可以直接讀取 Azure DevOps 工單，以自然語言描述需求，Skill 解析並讀取工單。工單內容包含：標題、狀態、指派、優先級、描述、Acceptance Criteria。

- Skill — `ado-devops`
- 操作 — 自然語言描述需求 → AI 解析工單
- 資訊 — 標題 / 狀態 / 指派 / 優先級 / 描述 / Acceptance Criteria

環節二：Wiki / 知識庫查詢

AI 在開發前先查詢相關的技術文件和架構決策。查詢維度：架構文件、最佳實務、API 文件、變更日誌。查詢結果用於在開發前了解既有約束，避免重複踩坑。

- 維度 — 架構文件 / 最佳實務 / API 文件 / 變更日誌
- 用途 — 了解既有約束，避免重複踩坑
- 範例 — 發現 HikariCP `maxPoolSize=10`，但有 20 個 instance

環節三：TDD 開發

使用 Agentic TDD 完成開發（第五、六堂內容）。核心節奏：🔴 寫失敗的測試 → 🟢 實作讓測試通過 → ♻️ 重構。

- 🔴 失敗測試 — 寫一個目前會失敗的測試明確定義需求
- 🟢 實作 — 寫最少的程式碼讓測試通過
- ♻️ 重構 — 改善程式碼結構但不改變外部行為

環節四：CI 自動測試

AI 自動執行單元測試、整合測試並產出覆蓋率報告。建議覆蓋率門檻 80% 以上。

- `mvn test` — 單元測試
- `mvn verify` — 整合測試
- `mvn jacoco:report` — 覆蓋率報告

環節五：AI Code Review

AI 在發 PR 前先自己做一次 Code Review。使用 PR Reviewer Skill，報告分三類：

✅ 通過 / ⚠️ 建議改善 / ❌ 必須修正。

- ✅ 通過 — 架構分層正確、測試完整、錯誤處理得當
- ⚠️ 建議改善 — 方法過長、建議移至 Configuration Properties

- **✗ 必須修正** — TODO comment 未處理、缺少 @Transactional

環節六：LLM Wiki 動態同步

開發完成後，AI 自動更新 Wiki 文件。新增變更日誌、更新技術文件，並在 A2P 審批後執行。

- **更新技術文件** — 連線池設定段落調整
- **新增變更日誌** — 連線池調校紀錄
- **更新 API 文件** — POST /api/v1/orders 說明

任務：

- □ 環節一：ADO 工單綁定 — 使用 ado-devops Skill 讀取工單
- □ 環節二：Wiki 查詢 — 查訂單服務架構文件、連線池設定規範
- □ 環節三：TDD 開發 — 參考第五、六堂的 Agentic TDD
- □ 環節四：CI 自動測試 — mvn test / verify / jacoco:report
- □ 環節五：AI Code Review — 使用 PR Reviewer Skill
- □ 環節六：LLM Wiki 動態同步 — 自動更新技術文件

常見疑問：

- **Q: 如果 Wiki 查不到相關文件怎麼辦？** A: 可以先做初步分析，然後在開發過程中建立文件，作為知識庫的一部分。
- **Q: CI 測試失敗時 AI 會怎麼處理？** A: AI 會分析錯誤訊息、修復測試或實作代碼、重新執行直到通過。這是 Agentic Loop 的一部分。



ADO 工單內容截圖

知識庫閉環概念

學習目標：

- 能夠解釋為何每次開發都會讓 AI 變得更聰明
- 能夠列舉三個形成閉環的關鍵行為
- 能夠判斷哪些知識適合放 Wiki

知識庫閉環：讓 AI 越來越聰明

每次開發都會產生新的知識（架構決策、API 設計、踩過的坑）。這些知識存入 Wiki 後，AI 下次開發時就能參考，形成正向循環。

形成閉環的三個關鍵行為

- **主動記錄** — 每次修復 bug，都同步更新 Wiki 的「已知問題」章節
- **引用積累** — 下次遇到類似問題，先查 Wiki 再動手
- **版本同步** — PR 合併後，自動更新相關技術文件

哪些知識適合放 Wiki

- **✅ 適合** — 架構決策與理由、API 設計原則、常見錯誤與解決方式、效能調校參數
- **❌ 不適合** — 純程式碼（留在 Git）、臨時性備註、涉及機密資料的內容

任務：

- □ 畫出知識庫閉環循環圖
- □ 說明 Wiki 文件的累積如何影響下次開發的品質
- □ 討論：哪些知識適合放 Wiki？哪些不適合？



知識庫閉環循環圖

Quick Win — 觸發 AI PR Reviewer

學習目標：

- 能夠建立一個含 Clean Architecture 違規的 Spring Boot 程式碼
- 能夠對該檔案執行 AI Code Review
- 能夠從報告中揪出全部 5 個架構違規

任務目標

建立一個故意違規的 Spring Boot 程式碼，然後對其執行 AI Code Review，觀察 AI 能否揪出全部 5 個 Clean Architecture 違規。

Note: 預期結果：AI 應抓出全部 5 個違規。✅ Controller 直接注入 EntityManager | ✅ Controller 直接執行 SQL | ✅ 沒有 DTO 轉換 | ✅ 使用

Step 1–4 教學

以下為課堂實作的四個步驟。學員實際操作，教師觀察並引導。

步驟動作預期產出

Step 1 mkdir pr-review-demo && git init 建立練習專案

Step 2 建立 BadController.java (含 5 個違規) 程式碼檔案已建立

Step 3 讓 AI Review (RTFC prompt) AI 輸出 Code Review 報告

Step 4 對照檢查清單確認 5/5 違規全部被抓到

BadController.java 範例 (含 5 個違規)

以下為示範用的錯誤程式碼。學員應自行建立並執行 AI Review。

行號 問題 程式碼 違規類型

11 @PersistenceContext private EntityManager em;  Controller 直接注入 EntityManager

13–17 em.createNativeQuery("INSERT INTO orders ...").executeUpdate(); 
Controller 直接執行 SQL

(整段) return body; // 直接回傳 Request Body  沒有 DTO 轉換

18 System.out.println("Order created: " + body);  使用 System.out.println (應用 Logger)

(整段) 無 try-catch 無 @ExceptionHandler  沒有錯誤處理

任務：

- □ 建立一個 BadController.java (含 5 個 Clean Architecture 違規)
- □ 對該檔案執行 AI Code Review
- □ 對照檢查清單確認全部 5 個違規都被抓到
- □ 修復後再次讓 AI Review 確認所有問題已解決



BadController.java 違規標注截圖

回家作業說明 — 結業挑戰

學習目標：

- 能夠獨立在真實低風險 Backlog 工單上完成從工單到 PR 的全流程
- 能夠填寫結業挑戰紀錄表

結業挑戰目標

證明你可以用 AI Agent 完成從工單到 PR 的全流程。選擇一張低風險的 Backlog 工單（bug fix 或小功能），確保不涉及核心業務邏輯或安全相關變更，並獲得主管同意後開始。

Note: ⚠️ 安全提醒：這是學習練習，請勿在正式 production 環境測試。

7 步驟流程

步驟工具/Skill產出

1. 讀取工單ado-devops工單內容 + AC
2. 查詢 Wiki相關技術文件
3. TDD 開發Agentic TDD通過的測試 + 實作
4. 測試通過mvn test覆蓋率報告
5. Code ReviewPR ReviewerReview 報告
6. 更新文件Wiki 已更新
7. 發 PRgitPR 連結

品質檢查清單

- ☒ — PR 的程式碼有對應的測試
- ☒ — 測試全部通過
- ☒ — 沒有跳過 A2P 審批
- ☒ — Wiki 文件已更新
- ☒ — commit message 符合規範

評分標準

項目比重

流程完整度（7 個步驟是否都執行） 30%

程式碼品質（測試、架構、規範） 25%

文件更新完整度15%

效率提升數據15%

心得反思的深度15%

任務：

- □ 挑選一張低風險 Backlog 工單（bug fix 或小功能）
- □ 完成 7 步驟流程並記錄截圖與耗時
- □ 填寫結業挑戰紀錄表
- □ 自評品質檢查：測試、架構、A2P、Wiki 更新、commit 規範

 結業挑戰紀錄表範本

七堂課總回顧

學習目標：

- 能夠將七堂課的內容串成一條主線
- 能夠找出自己最需要加強的環節
- 能夠說出「AI Coding Agent 三大紀律」

七堂課核心技能總表

堂數主題核心技能

1企業級 AI 基礎建設CLAUDE.md、架構邊界

2解構大廠 Agent 底層Agentic Loop、Error Recovery

3專屬 SRE 兵器庫MCP Server、AI-Friendly 腳本

4擴充 GitAgent 生態Skill 開發、防呆 Schema

5AI 系統分析實戰逆向工程、規格萃取

6AI 架構重構實戰Agentic TDD、Clean Architecture

7全自動化 AI SDLC端到端 Workflow、知識庫閉環

 AI Coding Agent 的三大紀律

這是七堂課的核心精華，牢記這三點可以讓你與 AI 協作時少走很多彎路。

- **架構先行** — 用 CLAUDE.md 定義邊界，讓 AI 在框架內發揮
- **規格驅動** — 先有規格再寫程式，測試是規格的可執行版本
- **人機協作** — AI 做重複性工作，人類做決策和審批

結業不是結束

把你學到的 Skill 貢獻到團隊 Repo，讓全團隊的 AI 都能受益。這才是 AI 時代的工程文化。建議：將你最常用的 RTFC prompt、Skill 配置、CLAUDE.md 模板整理成一個團隊共享的 AI 工具箱。

Note: 延伸學習：貢獻 Skill 到團隊 Repo、參與 AI Workshop 社群的知識分享、建立團隊內部的 AI Coding 規範文件。

任務：

- □ 填寫七堂課核心技能總表（可口頭討論或手寫）
- □ 背誦「AI Coding Agent 三大紀律」
- □ 討論：如何將學到的 Skill 貢獻到團隊 Repo



七堂課總回顧表格

七堂課總回顧

堂數主題核心技能

|---|---|---|

- 1企業級 AI 基礎建設CLAUDE.md、架構邊界
- 2解構大廠 Agent 底層Agentic Loop、Error Recovery
- 3專屬 SRE 兵器庫MCP Server、AI-Friendly 腳本
- 4擴充 GitAgent 生態Skill 開發、防呆 Schema
- 5AI 系統分析實戰逆向工程、規格萃取
- 6AI 架構重構實戰Agentic TDD、Clean Architecture
- 7全自動化 AI SDLC端到端 Workflow、知識庫閉環

 **AI Coding Agent 三大紀律**：架構先行 / 規格驅動 / 人機協作

 **結業不是結束**：把你學到的 Skill 貢獻到團隊 Repo，讓全團隊的 AI 都能受益。